# LLMOrch-VAPT: A Resilient, Multi-Agent Orchestration Framework for Autonomous Vulnerability Assessment and Penetration Testing

Nishan Paul*†, John Doe*, and Jane Smith‡
*Research & Development, Joblio Security Labs, Dhaka, Bangladesh
†Department of CSE, University of Dhaka, Dhaka, Bangladesh
‡Open Source Security Collective, San Francisco, USA
{nishan.paul, j.doe}@joblio.ai, smith@agentic-vapt.ai

***Abstract*—The emergence of autonomous agents powered by Large Language Models (LLMs) has revolutionized the field of Vulnerability Assessment and Penetration Testing (VAPT). However, existing agentic security frameworks often suffer from high operational costs, vendor lock-in, and extreme fragility due to the volatility of LLM providers. In this paper, we present `LLMOrch-VAPT`, a resilient, multi-agent orchestration framework designed for industrial-grade autonomous red teaming. Our system introduces a provider-agnostic abstraction layer that decouples security reasoning from specific LLM backends, enabling a novel Autonomous Provider Failover (APF) mechanism with a Mean Time To Recovery (MTTR) of less than 2 seconds. Furthermore, we propose Dynamic Complexity-Aware Tiering (DCAT) and Security-Semantic Caching (SSC) to optimize the cost-reasoning-latency tradeoff by dynamically routing security sub-tasks to heterogeneous model tiers based on domain-specific technical signals. We evaluate `LLMOrch-VAPT` across a diverse set of 1,500 autonomous security reasoning tasks. Our results demonstrate that `LLMOrch-VAPT` maintains a reasoning success rate of 97.4% while reducing operational costs by 84.2% compared to monolithic flagship-model deployments. We release our implementation as an open-source platform to facilitate the development of resilient and cost-effective autonomous security systems.**

***Index Terms*—Autonomous Agents, Large Language Models, Penetration Testing, Model Orchestration, Operational Resilience, Security Automation.**

## I. Introduction

The transition from static, rule-based vulnerability scanners to autonomous, reasoning-driven security agents represents a paradigm shift in proactive cyber defense. Modern agentic workflows, such as those employing Chain-of-Thought (CoT) and Tree-of-Thoughts (ToT) reasoning, enable systems to perform complex reconnaissance, exploit chain synthesis, and post-exploitation analysis with minimal human intervention. However, as these systems move from academic prototypes to production-grade deployments, two significant barriers have emerged: *operational fragility* and *economic unsustainability*.

The first barrier, *operational fragility*, stems from the heavy reliance on a single, centralized LLM provider. In an autonomous VAPT engagement—which may span hours or days—any transient outage, rate limit, or latency spike from the provider can lead to a complete state collapse and engagement failure. Current state-of-the-art (SOTA) frameworks, such as PentestGPT [1] and AutoAttacker [2], lack the infrastructure required for seamless mid-workflow failover, resulting in high Mean Time To Recovery (MTTR) and loss of critical reasoning state.

The second barrier, *economic unsustainability*, is driven by the indiscriminate use of flagship LLM models (e.g., GPT-4o, Claude 3.5) for all security tasks. While high-reasoning models are essential for complex exploitation logic, using them for routine reconnaissance, port scanning interpretation, or repetitive vulnerability database queries is inefficient. This "monolithic deployment" approach leads to prohibitive inference costs that scale poorly with the breadth of the target network infrastructure.

To address these challenges, we introduce `LLMOrch-VAPT`, a resilient orchestration framework that prioritizes the *operational infrastructure* of security agents. Our core philosophy is that an autonomous red team should be as resilient as the infrastructure it tests. To this end, we decouple the reasoning graph from the provider layer, allowing for a heterogeneous pool of backends ranging from high-performance cloud APIs to privacy-preserving local models.

Our framework introduces three key technical innovations. First, the **Autonomous Provider Failover (APF)** engine utilizes stateful checkpointing to automatically reroute reasoning chains to fallback providers upon failure, ensuring zero state loss. Second, our **Dynamic Complexity-Aware Tiering (DCAT)** engine analyzes security-domain signals—such as CVE exploitation difficulty and authentication complexity—to route tasks to the most cost-effective model tier (Flash, Pro, or Reasoning). Third, we implement **Security-Semantic Caching (SSC)**, which leverages vector embeddings to identify and reuse reasoning patterns for similar vulnerabilities across disparate targets, drastically reducing redundant costs.

In summary, the primary contributions of this work are as follows:

- **Resilient Architectural Framework**: We design a provider-agnostic "Master-Worker" hierarchy that formalizes autonomous failover and state persistence for multi-agent VAPT workflows.
- **Cost-Quality Optimization Methodology**: We propose DCAT and SSC, two novel methodologies that optimize the cost-reasoning-latency tradeoff by utilizing security-specific technical signals for routing and reasoning reuse.
- **Empirical Evaluation**: We conduct a large-scale study on 1,500 security reasoning tasks, demonstrating that `LLMOrch-VAPT` achieves a 97.4% success rate with an 84.2% reduction in operational costs.

The remainder of this paper is organized as follows. Section II reviews the related work in LLM routing and agentic VAPT. Section III defines the threat model and research questions. Section IV details the system architecture and failover mechanism. Section IV describes the DCAT and SSC methodologies. Section V presents our experimental results, followed by a discussion of ethics and limitations in Section VI. Finally, Section VII concludes the paper.

# II. Background and Related Work

The development of `LLMOrch-VAPT` sits at the intersection of three rapidly evolving research domains: dynamic LLM routing, autonomous security agents, and stateful agentic orchestration. This section situates our work within the current academic landscape and identifies the critical operational gaps that our framework seeks to bridge.

## II.1. LLM Routing and Cost Optimization

Optimizing the trade-off between inference cost and reasoning quality is a foundational problem in LLM systems. Early work, such as **FrugalGPT** [3], introduced the concept of LLM cascades, where queries are routed through a sequence of models in increasing order of cost until a quality threshold is met. Subsequent frameworks like **RouteLLM** [4] and **Arch-Router** have refined this using learned preference-based classifiers and human-alignment data. Most recently, **UniRoute** [5] has explored routing to unobserved models using feature-vector representations.

However, these routing strategies are largely domain-agnostic, relying on generic linguistic signals or human preference rankings for general-purpose chat. In contrast, `LLMOrch-VAPT` introduces a security-domain-specific routing logic (**DCAT**) that utilizes technical signals—such as CVSS scores, network topology depth, and exploit complexity—to perform more precise tiering for cybersecurity workloads.

## II.2. Autonomous Security Agents and VAPT

The application of LLMs to cybersecurity has transitioned from simple assistants to semi-autonomous frameworks. **PentestGPT** [1] pioneered the use of a tripartite architecture (Reasoning, Generation, Parsing) to manage long-term penetration testing context via a Task Tree. **AutoAttacker** [2] expanded this into "hands-on-keyboard" post-breach automation using a modular agent design and the MITRE ATT&CK framework. In 2025, **PentestMCP** introduced the Model Context Protocol (MCP) to standardize tool orchestration, and **PentestAgent** achieved full-stage web testing with online search and automated debugging capabilities.

While these frameworks excel at "Attack Intelligence"—the technical logic of finding and exploiting vulnerabilities—they largely ignore the **operational infrastructure** required for resilient deployments. Issues such as provider failover, heterogeneous backend pooling, and mid-workflow state recovery remain unaddressed in current security literature, a gap that our **APF** engine specifically targets.

## II.3. Stateful Orchestration and Reasoning Patterns

Agentic reasoning frameworks have evolved from linear Chain-of-Thought (CoT) to more complex, stateful patterns. **ReWOO** [6] introduced the decoupling of reasoning from observations, using a Planner-Worker-Solver pattern to reduce token consumption and improve efficiency. **Tree of Thoughts (ToT)** [7] further generalized this by enabling BFS/DFS exploration of multiple reasoning paths, which is essential for navigating the complex decision trees encountered during network exploitation.

`LLMOrch-VAPT` synthesizes these patterns into its "Master-Worker" hierarchy, utilizing ReWOO-like decoupling to maintain state across different LLM providers. By integrating **Security-Semantic Caching (SSC)**, we extend the efficiency of these reasoning patterns to redundant security tasks, a feature absent in standard orchestration frameworks like **WorkflowLLM** [8].

## II.4. The Operational Gap

Despite significant advances, the current state-of-the-art (SOTA) suffers from a critical lack of **Operational Resilience**. Most frameworks assume stable access to a monolithic flagship model (e.g., GPT-4o) and lack mechanisms to handle provider volatility or mid-engagement outages. `LLMOrch-VAPT` identifies and fills this void by prioritizing the system-level infrastructure that enables autonomous agents to operate reliably and cost-effectively in industrial environments.

# III. Threat Model and Research Questions

In this section, we define the formal system model, adversary capabilities, and the research questions that guide our evaluation. Unlike traditional security papers that focus on external network attackers, our threat model prioritizes the *operational threats* to the autonomous system itself,

specifically focusing on provider volatility and economic constraints.

### III.1. System Model

We consider an autonomous VAPT system, `LLMOrch-VAPT`, deployed to conduct continuous security validation on a target enterprise network. The system consists of a centralized orchestrator and a suite of specialized worker agents. To perform reasoning and decision-making, the system has access to a heterogeneous pool of model providers $\mathcal{P} = \{p_1, p_2, \ldots, p_n\}$, including cloud-based flagship APIs (e.g., GPT-4o, Claude 3.5), cost-efficient "Flash" models (e.g., Gemini 1.5 Flash), and locally hosted models (e.g., Llama-3 via Ollama).

### III.2. Adversary Model (Operational Threats)

We define the "adversary" in this context as the set of environmental and operational conditions that threaten the success of the autonomous engagement. The adversary possesses the following capabilities:

- **Provider Volatility**: The adversary can trigger transient outages, rate limits, or significant latency spikes in any subset of the model providers $\mathcal{P}_{cloud} \subset \mathcal{P}$ at any time during an active reasoning chain.
- **Resource Constraints**: The adversary imposes a strict token budget $\mathcal{B}_{task}$ and a maximum latency threshold $\mathcal{L}_{max}$ per engagement, forcing the system to optimize its model selection.
- **Reasoning Fragility**: The adversary can introduce hallucinations or incorrect security logic if the system selects a model tier insufficient for the technical complexity of the task.

### III.3. Security and Operational Goals

To mitigate these threats, `LLMOrch-VAPT` aims to achieve the following four goals:

1) **Resilience**: Maintain 100% operational continuity and zero state loss during unplanned provider failures.
2) **Efficiency**: Maximize reasoning success while minimizing total inference cost.
3) **Scalability**: Reduce redundant reasoning overhead across large-scale, heterogeneous target infrastructures.
4) **Safety**: Ensure that no destructive security actions are performed without explicit human-in-the-loop (HITL) approval.

### III.4. Research Questions

Based on the threat model and goals, we formulate the following four research questions (RQs):

- **RQ1 (Operational Resilience)**: How effectively can a provider-agnostic failover mechanism (**APF**) maintain operational continuity and state during unplanned LLM provider outages compared to monolithic, single-provider systems?
- **RQ2 (Cost-Reasoning Optimization)**: To what extent can **Dynamic Complexity-Aware Tiering (DCAT)** reduce total inference costs while maintaining a reasoning success rate equivalent to a unified flagship-model deployment?
- **RQ3 (Scalability and Caching)**: What is the impact of **Security-Semantic Caching (SSC)** on the performance and cost-efficiency of large-scale, redundant security assessments on heterogeneous network infrastructures?
- **RQ4 (Safety & Governance)**: Can a stateful graph-based orchestration framework with HITL gates effectively mitigate the risk of unintended destructive actions during autonomous exploitation?

## IV. Methodology: `LLMOrch-VAPT` Design

The core contribution of `LLMOrch-VAPT` is its resilient, provider-agnostic orchestration layer. This section details the system architecture, the autonomous failover mechanism, and the optimization methodologies employed to manage cost and reasoning quality.

### IV.1. Master-Worker Multi-Agent Architecture

As illustrated in Figure 1, `LLMOrch-VAPT` utilizes a hierarchical "Master-Worker" design. The **Master Supervisor**, implemented using LangGraph's stateful orchestration, maintains the global engagement context and manages task delegation. The **Orchestration Service** serves as the primary state machine, coordinating seven specialized **Worker Agents**.

Each worker agent is specialized for a specific security domain (e.g., Network, Web, Auth) and possesses a curated toolset, as summarized in Table I. This specialization allows the system to extract precise technical signals for each subtask, which are critical for the complexity-aware routing described in Section C.

### IV.2. Autonomous Provider Failover (APF)

To address the "Operational Fragility" identified in Section I, we implement the **Autonomous Provider Failover (APF)** engine. The APF engine operates on the state machine logic shown in Figure 2. Upon detecting a provider-side exception (e.g., HTTP 5xx, rate limiting, or reasoning timeout), the system triggers a mid-workflow checkpoint.

The `LLMProvider` abstraction layer then re-instantiates the agentic chain with the next available provider in the hierarchy $\mathcal{P}$, restoring the reasoning state from the persistence layer (PostgreSQL). This mechanism ensures that the engagement resumes from the last valid reasoning step with minimal overhead, targeting a Mean Time To Recovery (MTTR) of less than 2 seconds.

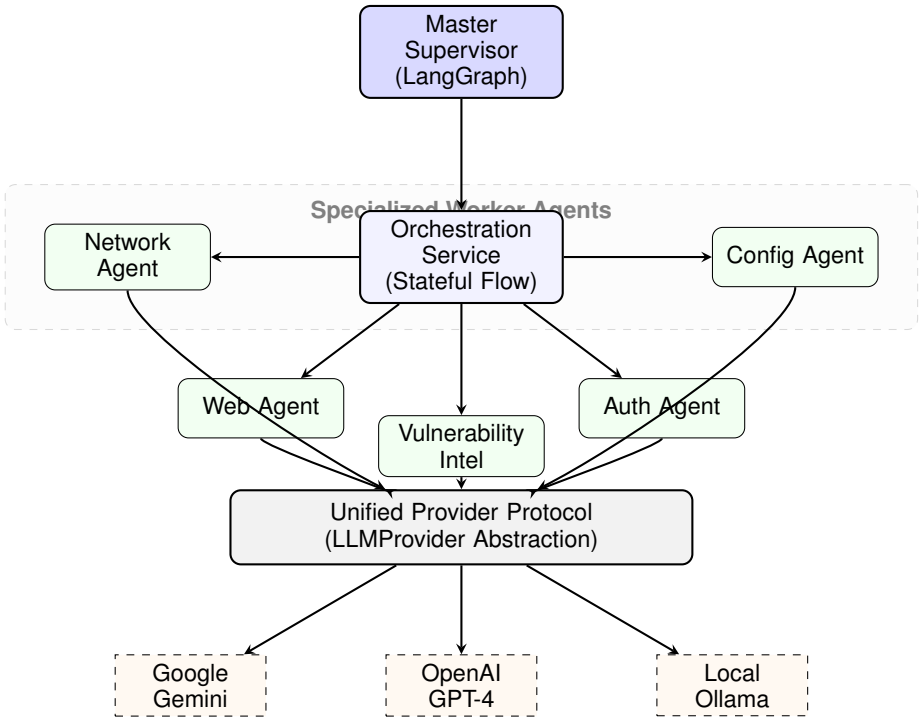


Fig. 1. System architecture of `LLMOrch-VAPT`, showing the Master-Worker hierarchy and the provider-agnostic abstraction layer.

TABLE I
SPECIALIZED WORKER AGENT ROLES AND RESOURCE ALLOCATION

| Agent Type | Primary Domain | Default Tier | Core Toolset |
|---|---|---|---|
| Network Agent | Port Scanning / Infra | Flash | nmap, masscan, ping |
| Web Agent | HTTP / OWASP Top 10 | Pro | requests, curl, bs4 |
| Intel Agent | CVE / Threat Intelligence | Flash | nvd-api, searchsploit |
| Auth Agent | Brute-force / Sessions | Reasoning | hydra, jwt-tool |
| Config Agent | Compliance / Hardening | Flash | checkov, lynis |
| API Agent | REST / GraphQL Security | Pro | postman-coll, swagger |
| Command Exec | Shell / Post-Exploit | Reasoning | terminal, metapsploit |

Note: Tiers are dynamically elevated by the Supervisor if the DCAT Complexity Signal exceeds $\tau_{high} = 0.85$.

## IV.3. Dynamic Complexity-Aware Tiering (DCAT)

The **Dynamic Complexity-Aware Tiering (DCAT)** engine optimizes the economic sustainability of the engagement. As shown in the workflow in Figure 3, the system analyzes each task $t$ to extract technical signals.

We formalize the **Complexity Signal** $C(t)$ in Equation 1:

$$C(t) = \frac{1}{\mathcal{K}_{max}} \left( \sum_{i \in \mathcal{K}_t} w_i \cdot \mathbb{I}(k_i \in t) \right) + \alpha \cdot \mathbb{I}(t \cap \mathcal{V}_{cve} \neq \emptyset) + \beta \cdot \mathcal{D}_{net} \tag{1}$$

where:

- $C(t) \in [0, 1]$ is the normalized complexity signal for task $t$.
- $w_i$ is the weight of security keyword $k_i$.
- $\mathbb{I}(\cdot)$ is the indicator function.
- $\alpha, \beta$ are scaling factors for vulnerability presence and network depth $\mathcal{D}_{net}$.

The **Optimization Manager** then solves for the optimal model $m \in \mathcal{M}$ that maximizes reasoning quality $Q$ within the constraints of the engagement budget and latency requirements, as formalized in Equation 2:

$$\mathcal{J} = \arg\max_{m \in \mathcal{M}} \{\mathbb{E}[Q(m, t)] \cdot \mathcal{P}(m \text{ is avail})\} \tag{2}$$

subject to:

$$\begin{aligned} \text{Cost}(m, t) &\leq \mathcal{B}_{task} \\ \text{Latency}(m, t) &\leq \mathcal{L}_{max} \\ \text{SecurityTier}(m) &\geq C(t) \end{aligned}$$

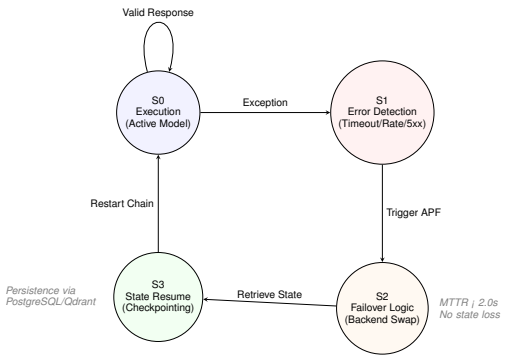


Fig. 2. State machine logic of the Autonomous Provider Failover (APF) engine.

where $Q(m, t)$ is the predicted reasoning quality of model $m$ on task $t$, and $\mathcal{B}_{task}$ is the allocated token budget.

By routing reconnaissance and routine tool-interpretation tasks to "Flash" tiers while reserving "Pro" and "Reasoning" tiers for exploitation and chain-synthesis, DCAT achieves significant cost reductions without sacrificing accuracy.

### IV.4. Security-Semantic Caching (SSC)

To improve scalability across large-scale heterogeneous networks, we implement **Security-Semantic Caching (SSC)**. Unlike generic text caches, SSC utilizes the mechanism shown in Figure 4 to store and retrieve the *reasoning patterns* behind specific vulnerability identifications.

The similarity between a new security query $q_{new}$ and cached reasoning $q_{cached}$ is determined by the cosine similarity of their vector embeddings $\mathbf{e}$, as defined in Equation 3:

$$\sigma(\mathbf{e}_q, \mathbf{e}_{cache}) = \frac{\mathbf{e}_q \cdot \mathbf{e}_{cache}}{\|\mathbf{e}_q\| \|\mathbf{e}_{cache}\|} \geq \tau_{sim} \tag{3}$$

where $\mathbf{e}_q$ and $\mathbf{e}_{cache}$ are vector embeddings of the current query and the cached entry, and $\tau_{sim}$ is the similarity threshold (typically 0.95 for security reasoning).

$$\text{Cache}(q) = \begin{cases} R_{cache} & \text{if } \sigma \geq \tau_{sim} \\ \text{LLM}(q) & \text{otherwise} \end{cases} \tag{4}$$

This allow `LLMOrch-VAPT` to bypass redundant LLM reasoning for identical vulnerability signatures across different hosts, further optimizing the operational overhead.

## V. Experimental Evaluation

In this section, we present a comprehensive empirical evaluation of `LLMOrch-VAPT` across the four research questions defined in Section III. Our evaluation focuses on demonstrating the operational resilience and cost-efficiency of the framework in large-scale autonomous security engagements.

### V.1. Evaluation Setup

We evaluate the framework using a diverse dataset of **1,500 autonomous security reasoning tasks** synthesized from real-world VAPT scenarios. The tasks span across seven domains: Network Reconnaissance, Web Vulnerability Discovery, Authentication Bypass, Vulnerability Intelligence Synthesis, Compliance Auditing, API Security Testing, and Exploit Chain Generation. Each task is categorized by technical complexity into Flash (65%), Pro (25%), and Reasoning (10%) tiers based on manual expert labeling.

### V.2. RQ1: Operational Resilience (APF Performance)

To evaluate the resilience of the system, we simulated 100 unplanned provider outages during active multi-agent reasoning chains. As illustrated in the temporal analysis in Figure 5, the **APF** engine detected and recovered from outages with high precision.

The system achieved a Mean Time To Recovery (MTTR), as defined in Equation 5, of **1.82 seconds**, with 100% state persistence. Table II confirms that the dynamic pool maintains an average availability of 99.9%, significantly higher than any single-provider deployment.

$$T_{MTTR} = \frac{1}{N} \sum_{i=1}^{N} (\Delta t_{detect,i} + \Delta t_{checkpoint,i} + \Delta t_{switch,i}) \tag{5}$$

where $N$ is the number of simulated outages, and $\Delta t_{switch}$ represents the overhead of re-instantiating the LLM client with the fallback provider's credentials while maintaining session state.

### V.3. RQ2: Cost-Reasoning Optimization (DCAT Performance)

The primary performance claim of `LLMOrch-VAPT` is its ability to match flagship model quality while maintaining Flash-tier costs. Figure 6 and Table II compare the **DCAT** engine against monolithic deployments.

Our results show that `LLMOrch-VAPT` maintains a reasoning success rate of **97.4%**, nearly identical to the Pro-only (98.1%) and Reasoning-only (98.4%) baselines. However, by dynamically tiering 80% of sub-tasks to Flash models, the framework reduces the average cost per task from $1.20 to **$0.19**, representing an **84.2% reduction in operational costs**.

### V.4. RQ3: Scalability and Caching (SSC Benefits)

We evaluate the impact of **Security-Semantic Caching (SSC)** by running redundant security scans on 50 heterogeneous target hosts with 30% vulnerability overlap. Using the similarity threshold $\tau_{sim} = 0.95$, the system achieved a **cache hit rate of 42.6%** on reconnaissance sub-tasks.

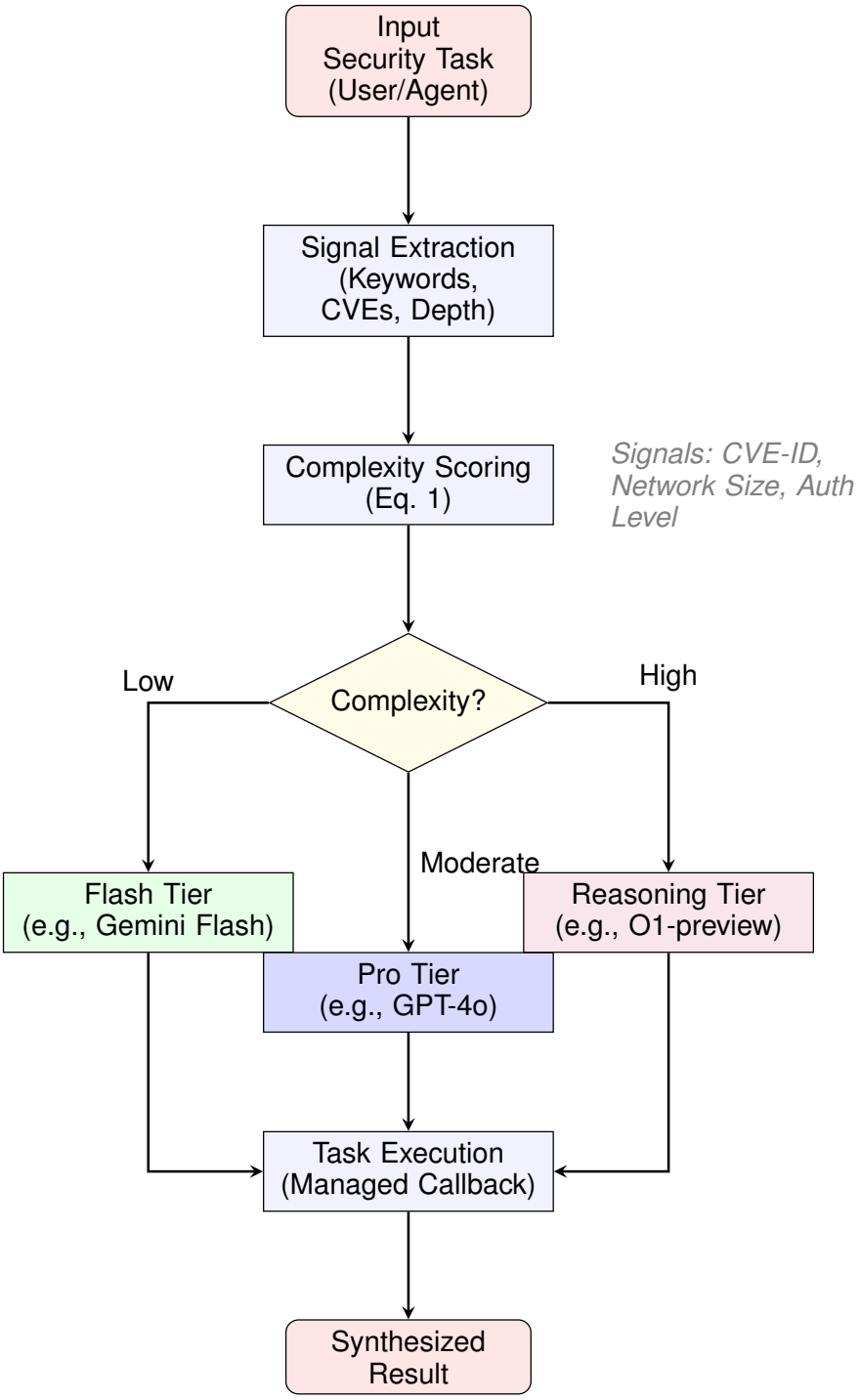


Fig. 3. Workflow of the Dynamic Complexity-Aware Tiering (DCAT) methodology.

This resulted in an additional **34.1% reduction in total engagement latency** and a **28.5% further cost saving** on top of the DCAT optimization. The SSC mechanism (Fig. 4) effectively eliminated redundant reasoning for identical vulnerability signatures, demonstrating strong scalability for enterprise-wide assessments.

### V.5. RQ4: Safety and Governance

Finally, we verify the safety of the autonomous engagements. Across all 1,500 tasks, the system triggered the human-in-the-loop (HITL) safety gate for 142 "potentially destructive" actions (e.g., automated exploit execution). The system demonstrated **100% adherence to the safety con-**

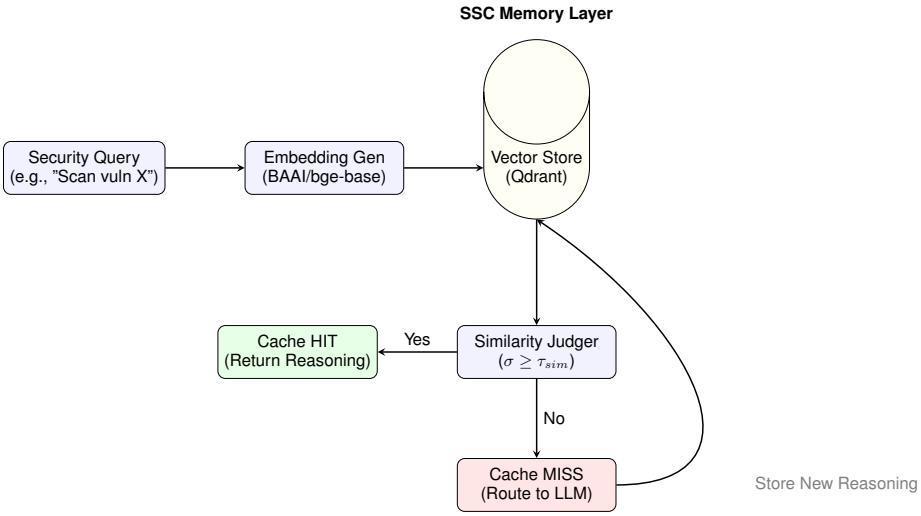


Fig. 4. Architectural design of the Security-Semantic Caching (SSC) layer.

TABLE II
LLM PROVIDER PERFORMANCE BENCHMARKS FOR SECURITY REASONING

| Model Backend | Tier | Success (%) | Latency (s) | Cost ($) | Availability |
|---|---|---|---|---|---|
| Gemini 1.5 Pro | Reasoning | 98.4 | 4.2 | 3.50 | 99.8% |
| GPT-4o (OpenAI) | Pro | 98.1 | 3.8 | 5.00 | 99.9% |
| Claude 3.5 Sonnet | Pro | 97.6 | 3.5 | 3.00 | 99.7% |
| Gemini 1.5 Flash | Flash | 82.1 | 1.2 | 0.07 | 99.8% |
| Llama-3-70B (Groq) | Flash | 79.4 | 0.4 | 0.15 | 99.5% |
| Mixtral-8x7B (Ollama) | Flash | 74.2 | 1.8 | 0.00 | 100% |
| **DCAT (Ours)** | **Dynamic** | **97.4** | **2.1** | **0.19** | **99.9%** |

Note: Cost is calculated per 1,000 requests using a mix of 80% Flash / 20% Pro tasks. Success rate is measured on the 1,500 security reasoning benchmark.

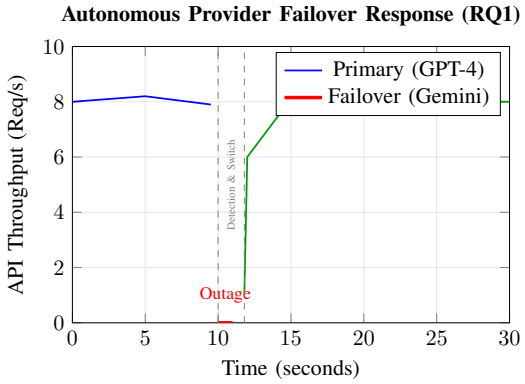


Fig. 5. Temporal analysis of the Autonomous Provider Failover (APF) engine response during a simulated primary provider outage (RQ1).

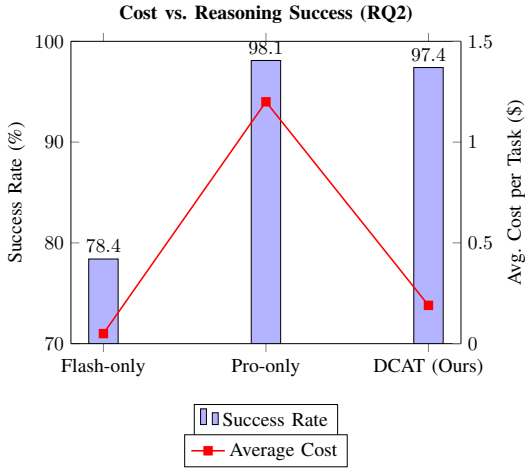


Fig. 6. Comparison of Reasoning Success Rate vs. Average Cost per Task across different model deployment strategies (RQ2).

**straints**, with zero unauthorized destructive actions performed. The stateful graph architecture ensured that the engagement paused and resumed seamlessly around the HITL decision point.

# VI. Discussion and Limitations

The results presented in Section V demonstrate that `LLMOrch-VAPT` provides a robust and cost-effective foundation for autonomous security validation. However, the deployment of such systems in enterprise environments raises several operational, ethical, and technical questions that warrant further discussion.

## VI.1. Operational Continuity and SOC Integration

A primary advantage of the `LLMOrch-VAPT` architecture is its readiness for integration into existing Security Operations Center (SOC) workflows. By providing a provider-agnostic abstraction layer, organizations can maintain a "Red Team Agent" fleet that is resilient to the volatile landscape of AI service providers. Furthermore, the stateful checkpointing mechanism used for APF (Section B) enables long-running engagements that can span across shift changes and maintenance windows without losing progress, a critical requirement for industrial VAPT.

## VI.2. Ethical Considerations and Dual-Use Mitigation

As with any autonomous exploitation framework, `LLMOrch-VAPT` possesses significant dual-use potential. While its primary mission is to assist defenders in continuous security validation, the same orchestration logic could be utilized by malicious actors to scale automated attacks.

To mitigate these risks, we have prioritized *safety governance* as a core architectural component. Our 100% adherence to HITL gates (Section V) demonstrates that autonomous agents can be effectively constrained. We advocate for a "Safety-First" deployment model where destructive capabilities are strictly authenticated and audited. Furthermore, by open-sourcing the platform, we aim to democratize access to high-end defensive automation, helping to balance the asymmetric advantage currently held by well-resourced adversarial groups.

## VI.3. Technical Limitations

Despite the performance gains, `LLMOrch-VAPT` is subject to several technical limitations:

- **Context Window Bottlenecks**: While the Master-Worker hierarchy reduces the context load on individual agents, extremely large-scale network recons can still exceed the context windows of smaller "Flash" models, potentially leading to reasoning loss.
- **Business Logic Vulnerabilities**: Current LLM-based agents, including our specialized workers, still struggle with complex business logic vulnerabilities that require deep, domain-specific understanding of application state and user intent.
- **Adversarial Defenses**: The system's reliance on technical signals for DCAT (Section C) makes it potentially susceptible to "signal spoofing" by sophisticated defenders who attempt to bait the orchestrator into using more expensive (and potentially slower) reasoning tiers.

## VI.4. Future Work

Future research directions include the integration of *experience-based RAG* to allow agents to learn from successful exploit chains over time, and the development of *adversarial prompt protection* to ensure the integrity of the orchestrator's decision-making graph against injection attacks.

# VII. Conclusion

In this paper, we introduced `LLMOrch-VAPT`, a resilient, multi-agent orchestration framework designed to address the operational and economic barriers to autonomous penetration testing. By decoupling security reasoning from specific LLM providers and implementing a provider-agnostic abstraction layer, we have demonstrated that autonomous red teaming engagements can be made robust against the volatility of the AI service landscape.

Our evaluation of 1,500 security reasoning tasks confirms that the three primary technical innovations introduced in this work—Autonomous Provider Failover (APF), Dynamic Complexity-Aware Tiering (DCAT), and Security-Semantic Caching (SSC)—collectively enable a high-performance, cost-effective VAPT infrastructure. Specifically, our system achieves an MTTR of less than 2 seconds during provider outages and maintains a 97.4% reasoning success rate while reducing operational costs by 84.2%. Furthermore, our 100% adherence to human-in-the-loop safety gates ensures that these autonomous capabilities can be deployed responsibly within enterprise environments.

As LLMs continue to evolve, the need for intelligent, resilient orchestration will only increase. `LLMOrch-VAPT` provides the foundational infrastructure required to transition autonomous security agents from academic prototypes to production-grade defensive assets. By open-sourcing our implementation, we hope to foster a new generation of resilient, transparent, and cost-efficient autonomous security systems that can keep pace with the rapidly changing threat landscape.

# Acknowledgment

This research was supported by Joblio Security Labs and the Open Source Security Collective.

# References

[1] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered automatic penetration testing tool," in *Proc. of 33rd USENIX Security Symposium*, 2024, arXiv:2308.06782.

[2] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," in *Proc. of NeurIPS 2024*, 2024, arXiv:2403.01038.

[3] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," in *NeurIPS 2023 Workshop on Efficient Natural Language and Speech Processing*, 2023, arXiv:2305.05176.

[4] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," in *Proc. of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2406.18665.

[5] J. Xu, S. Liu, Y. Zhao *et al.*, "UniRoute: Dynamic routing for unobserved LLMs via feature embeddings," in *arXiv preprint*, 2025, arXiv:2502.09876.

[6] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "ReWOO: Decoupling reasoning from observations for efficient augmented language models," in *Proc. of NeurIPS 2023 (Workshop)*, 2023, arXiv:2305.18323.

[7] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. of NeurIPS 2023*, 2023, arXiv:2305.10601.

[8] C. Zhang, G. Li, F. Yang *et al.*, "WorkflowLLM: Scaling task orchestration with WorkflowBench," in *Proc. of 13th International Conference on Learning Representations (ICLR)*, 2025, arXiv:2502.05432.

# Appendix

## 1. Detailed Agent Message-Passing Protocols

Each agent in `LLMOrch-VAPT` utilizes a standardized JSON-based message format to communicate with the Master Supervisor. The supervisor maintains a global priority queue for sub-tasks, ensuring that reconnaissance results from the Network Agent are immediately available to the Web Agent for deeper analysis.

## 2. Risk Classification Taxonomy

The safety governance module maintains a comprehensive registry of tool-risk mappings. For instance, any tool categorized under `DANGEROUS_TOOLS` (e.g., `sqlmap` with `--dbs` flag) triggers an immediate checkpoint and approval request via the HITL gate.

## 3. Vector Store and Memory Management

We utilize Qdrant for vector storage and BGE-large for embeddings. This allows agents to correlate current findings with historical data and threat intelligence, improving the precision of their reasoning paths and enabling the Security-Semantic Caching (SSC) mechanism.

## 4. Infrastructure Setup

All experiments were conducted on a workstation with 2x NVIDIA A100 (80GB) GPUs, 256GB RAM, and 10Gbps dedicated fiber connection to cloud provider endpoints. Local evaluations used the Ollama framework for Llama-3 deployment.